Titulo del Trabajo: Aprendizaje por Refuerzo

Francisco Jesús Camacho Molina
franciscojesus.camachom@um.es

Rodrigo Ruiz Herraiz
rodrigo.ruizh@um.es

Adrian Muñoz Jaime
adrian.munozj@um.es

15 de marzo de 2026

Índice general

1. Problema del Bandido de k-brazos **3**

1.1. Introducción . . . 3

1.2. Marco Teórico y Formulación del Problema . . . 3

1.2.1. Definición Formal del Entorno . . . 3

1.2.2. El Concepto de Regret y la Cota de Lai-Robbins . . . 3

1.2.3. Análisis de Distribuciones y Efectos de Varianza . . . 4

1.3. Algoritmos Implementados . . . 4

1.3.1. Familia ϵ-Greedy y la Corrección de la Inicialización . . . 4

1.3.2. Algoritmo UCB1 (Upper Confidence Bound) . . . 4

1.3.3. Algoritmo Softmax y el Truco de Estabilidad Numérica . . . 5

1.4. Evaluación, Configuración Experimental y Metodología . . . 5

1.4.1. Parámetros de la Simulación . . . 5

1.4.2. Métricas de Rendimiento . . . 5

1.5. Análisis de Resultados . . . 5

1.5.1. Dinámica Exploratoria de ϵ-Greedy . . . 5

1.5.2. El Fenómeno de la Escala y el Regret Asintótico . . . 6

1.5.3. Sensibilidad a la Distribución: UCB1 vs Softmax . . . 6

1.6. Discusión y Conclusiones . . . 7

2. Aprendizaje en Entornos Complejos **8**

2.1. Introducción . . . 8

2.2. Desarrollo . . . 8

2.2.1. Entorno discreto: FrozenLake-v1 . . . 8

2.2.2. Entorno continuo: CartPole-v1 . . . 8

2.2.3. Trabajos Relacionados y Contexto . . . 8

2.3. Algoritmos . . . 9

2.3.1. Métodos Tabulares de Monte Carlo . . . 9

2.3.2. Métodos Tabulares de Diferencias Temporales (TD) 10
2.3.3. Métodos con Aproximación de Funciones 10
2.4. Evaluación y Experimentos 11
2.4.1. Métodos Tabulares en FrozenLake-v1 11
2.4.2. Métodos Aproximados en CartPole-v1 13
2.5. Conclusiones 14

Capítulo 1

Problema del Bandido de k-brazos

1.1. Introducción

El aprendizaje por refuerzo (RL, por sus siglas en inglés) basa su enfoque en la interacción continua de un agente con un entorno para aprender a tomar decisiones óptimas a través de la experiencia. Dentro de este vasto campo, el problema del bandido de k-brazos (o *Multi-Armed Bandit*) modela de forma simplificada una situación de toma de decisiones secuencial bajo incertidumbre. Al carecer de estados cambiantes, constituye el escenario puro y fundamental para comprender el dilema más importante del RL: el equilibrio entre la exploración y la explotación.

La motivación principal de este trabajo reside en que los mejores agentes inteligentes son aquellos capaces de encontrar la frontera óptima entre probar nuevas opciones (exploración) para descubrir recompensas potencialmente mayores, y capitalizar el conocimiento ya adquirido con alta certeza (explotación) para maximizar la ganancia acumulada a corto y medio plazo. Este paradigma matemático tiene aplicaciones directas, críticas y altamente rentables en escenarios del mundo real. Ejemplos de ello son los ensayos clínicos de tratamientos médicos (donde la vida de los pacientes depende de identificar rápidamente el mejor fármaco), los test A/B en el diseño de interfaces web, la optimización algorítmica de campañas de publicidad online o el ajuste en tiempo real de los sistemas de recomendación en plataformas de *streaming*.

Los objetivos específicos que persigue este informe de investigación son los siguientes:

- Completar, formalizar y analizar el comportamiento de la familia de algoritmos ϵ-greedy. Se hará especial hincapié en corregir sus deficiencias estructurales en casos límite, como el estancamiento asintótico que sufre la variante ($\epsilon = 0$), mediante la implementación de fases de inicialización obligatoria.
- Implementar y evaluar algoritmia avanzada que supera la exploración ciega: por un lado, métodos deterministas guiados por la incertidumbre y basados en intervalos de confianza (algoritmo UCB1); por otro, métodos estocásticos basados en la distribución térmica de Boltzmann y preferencias de valor (algoritmo Softmax).
- Contrastar de forma teórica y empírica cómo la escala de la recompensa y la varianza intrínseca del entorno afectan a la velocidad de aprendizaje del agente. Para ello, se utilizarán tres distribuciones subyacentes bien diferenciadas: Normal (continua), Bernoulli (señal débil discreta) y Binomial (señal fuerte discreta).
- Realizar un análisis crítico de las métricas resultantes, evaluando la recompensa promedio, el porcentaje de convergencia hacia la acción óptima y, fundamentalmente, el crecimiento del *regret* acumulado, comparándolo con la cota teórica logarítmica descrita en la literatura científica.

El documento se organiza de la siguiente manera: la Sección 2 define formalmente el problema, el concepto de *regret* y desglosa matemáticamente las distribuciones utilizadas. La Sección 3 detalla los algoritmos y las soluciones de estabilidad numérica implementadas. La Sección 4 presenta el diseño experimental y el análisis profundo de los resultados. Finalmente, la Sección 5 expone las conclusiones extraídas del estudio.

1.2. Marco Teórico y Formulación del Problema

1.2.1. Definición Formal del Entorno

El problema del bandido de k-brazos asume un entorno estacionario modelado como una máquina tragaperras (*slot machine*) que dispone de un conjunto de acciones $\mathcal{A} = \{1, 2, \ldots, k\}$. El proceso se desarrolla en pasos temporales discretos $t = 1, 2, \ldots, T$.

En cada paso t, el agente selecciona una acción $A_t \in \mathcal{A}$ y el entorno le devuelve una recompensa estocástica $R_t \sim \mathcal{P}_{A_t}$, donde $\mathcal{P}_i$ es una distribución de probabilidad subyacente que es totalmente desconocida a priori para el agente. Cada acción a tiene asociado un valor esperado verdadero o recompensa media teórica, denotada como:

$$q_*(a) = \mathbb{E}[R_t \mid A_t = a] \tag{1.1}$$

Si el agente conociera el valor exacto de $q_*(a)$ para todos los brazos, el problema sería trivial: siempre seleccionaría el brazo óptimo $a^* = \arg\max_a q_*(a)$. Sin embargo, al existir incertidumbre, el agente debe mantener estimaciones empíricas $Q_t(a) \approx q_*(a)$ basadas en el historial de recompensas observadas.

1.2.2. El Concepto de Regret y la Cota de Lai-Robbins

El objetivo central del agente es maximizar la suma de recompensas observadas. Matemáticamente, optimizar esta métrica es estrictamente equivalente a minimizar el rechazo o *regret* acumulado $R(T)$.

El *regret* se define como la pérdida de recompensa esperada por no haber elegido el brazo óptimo en cada uno de los instantes de tiempo. Se puede formular como:

$$R(T) = \sum_{t=1}^{T} (q_*(a^*) - \mathbb{E}[R_t \mid A_t]) \tag{1.2}$$

Definiendo la brecha de recompensa o *gap* de una acción subóptima como $\Delta_a = q_*(a^*) - q_*(a)$, y denotando $\mathbb{E}[N_T(a)]$ como el número esperado de veces que el agente selecciona el brazo a en T pasos, el *regret* también puede expresarse como:

$$R(T) = \sum_{a \neq a^*} \Delta_a \cdot \mathbb{E}[N_T(a)] \tag{1.3}$$

Esta formulación revela que un buen algoritmo debe minimizar el número de veces que visita brazos subóptimos. En 1985, los investigadores Lai y Robbins establecieron un teorema fundamental: demostraron que para cualquier algoritmo consistente, el *regret* acumulado está acotado inferiormente por una función logarítmica. Específicamente, a medida que $T \to \infty$:

$$\lim_{T\to\infty} \inf \frac{R(T)}{\ln(T)} \geq \sum_{a \neq a^*} \frac{\Delta_a}{D_{KL}(\mathcal{P}_a \parallel \mathcal{P}_{a^*})} \tag{1.4}$$

Donde D_{KL} es la divergencia de Kullback-Leibler entre la distribución del brazo subóptimo y el óptimo. En la práctica, esto significa que el mejor comportamiento asintótico posible que podemos esperar de un algoritmo es un crecimiento $\mathcal{O}(\ln T)$. Aquellos algoritmos que logran este comportamiento (como UCB) se consideran asintóticamente eficientes.

1.2.3. Análisis de Distribuciones y Efectos de Varianza

Para evaluar la robustez algorítmica ante diferentes niveles de ruido, se modelaron tres distribuciones subyacentes. La Tabla 1.1 resume las características clave de los entornos discretos estudiados.

Tabla 1.1: Comparativa teórica entre los entornos discretos estudiados

Aspecto Analizado	Entorno Bernoulli	Entorno Binomial ($n = 100$)
Rango de recompensa	$\{0, 1\}$	$\{0, 1, \ldots, 100\}$
Esperanza óptima $q_*(a^*)$	ej. 0,861	ej. 79,3
Varianza máxima teórica	$\sigma^2 = p(1-p) \leq 0{,}25$	$\sigma^2 = np(1-p) \leq 25$
Brecha de recompensa (Δ_a)	Décimas (ej. $\Delta \approx 0{,}05$)	Decenas (ej. $\Delta \approx 10{,}0$)
Naturaleza de la señal	Débil (alta dispersión)	Fuerte (clara separación)

- **Distribución Normal:** La recompensa de cada brazo sigue $R_i \sim \mathcal{N}(\mu_i, \sigma^2 = 1)$. Proporciona recompensas continuas. Al tener varianza unitaria conocida, es el entorno estándar para evaluar la convergencia base.
- **Distribución Bernoulli:** La recompensa sigue $R_i \sim \mathcal{B}(1, p_i)$. Modela eventos binarios estrictos (ej. un usuario hace clic o no en un anuncio publicitario). Todo el espacio de recompensas está confinado al conjunto $\{0, 1\}$. Esto implica que cada interacción aporta una cantidad mínima de información al agente, generando estimaciones altamente volátiles en las primeras etapas y constituyendo el entorno más difícil de resolver.
- **Distribución Binomial:** Sigue $R_i \sim Bin(n = 100, p_i)$. Modela la suma de n ensayos de Bernoulli independientes (ej. la tasa de éxito de una campaña de *marketing* enviada a un lote de 100 usuarios simultáneos). Aunque la varianza absoluta de la distribución crece numéricamente, la gran separación entre las esperanzas matemáticas facilita enormemente la discriminación por parte de la IA.

1.3. Algoritmos Implementados

Una decisión arquitectónica fundamental en este estudio ha sido la inclusión de una **fase de inicialización** obligatoria para los tres algoritmos. Esta fase fuerza al agente a seleccionar cada uno de los k brazos exactamente una vez durante los primeros $t \leq k$ pasos temporales.

1.3.1. Familia ϵ-Greedy y la Corrección de la Inicialización

Es una estrategia heurística clásica que decide explotar la mejor estimación con probabilidad $1 - \epsilon$, y explorar uniformemente al azar con probabilidad ϵ. La regla de selección se define como:

$$A_t = \begin{cases} \text{Acción aleatoria uniforme } \in \mathcal{A}, & \text{con probabilidad } \epsilon \\ \arg\max_a Q_t(a), & \text{con probabilidad } 1-\epsilon \end{cases} \tag{1.5}$$

La fase de inicialización es de vital importancia matemática para el caso particular de $\epsilon = 0$ (*Greedy* puro). Si no se implementa, y el agente selecciona un brazo al azar en $t = 1$ obteniendo una recompensa positiva, el valor empírico de ese brazo será mayor que 0 (el valor por defecto de los demás). Al ser una política puramente explotadora, el agente nunca más seleccionará otro brazo, quedándose atascado infinitamente en una solución potencialmente subóptima.

1.3.2. Algoritmo UCB1 (Upper Confidence Bound)

UCB1 abandona la exploración aleatoria en favor de un enfoque determinista basado en el principio de "optimismo ante la incertidumbre". Utiliza la desigualdad de Hoeffding, un teorema estadístico que acota la probabilidad de que la media empírica de variables aleatorias se desvíe de su media real: $\mathbb{P}(|\bar{X} - \mu| \geq u) \leq 2e^{-2nu^2}$.

Partiendo de este principio, UCB1 calcula un límite superior de confianza (el *Upper Bound*) para cada brazo. La selección se formula como:

$$A_t = \arg\max_a \left[Q_t(a) + c \cdot \sqrt{\frac{\ln t}{N_t(a)}} \right] \tag{1.6}$$

El primer término es la explotación (la media empírica). El segundo término es el bono de exploración, ponderado por el hiperparámetro $c > 0$. Este bono crece logarítmicamente a medida que avanza el tiempo total t, pero decrece radicalmente cada vez que el brazo específico es seleccionado (ya que $N_t(a)$ está en el denominador). Esto garantiza que todos los brazos se exploren lo suficiente, garantizando asintóticamente la cota $\mathcal{O}(\ln T)$ de Lai y Robbins.

1.3.3. Algoritmo Softmax y el Truco de Estabilidad Numérica

En lugar de explorar uniformemente (como ϵ-greedy), Softmax gradúa la probabilidad de explorar un brazo en función de su mérito actual. Asigna probabilidades de selección basadas en la distribución de Boltzmann, controladas por un hiperparámetro de temperatura τ:

$$P(A_t = a) = \frac{e^{Q_t(a)/\tau}}{\sum_{b=1}^{k} e^{Q_t(b)/\tau}} \tag{1.7}$$

Si $\tau \to \infty$, las exponenciales tienden a $e^0 = 1$, resultando en exploración aleatoria uniforme. Si $\tau \to 0$, la diferencia entre los valores se magnifica infinitamente, tendiendo a una política *greedy*.

Solución a la inestabilidad computacional (*Overflow*): Durante el desarrollo, se detectó un problema crítico en entornos de señal fuerte como el Binomial ($n = 100$). En este entorno, un brazo puede tener $Q_t(a) \approx 80$. Si se utiliza una temperatura de $\tau = 0{,}1$, el algoritmo intentaría calcular matemáticamente e^{800}. Dado que el límite de representación de coma flotante estándar de 64 bits en la mayoría de arquitecturas informáticas ronda e^{709}, esto provoca un error fatal de *overflow* en memoria.

Para garantizar la estabilidad numérica, la implementación aplica una propiedad matemática de las exponenciales: restar una constante a todos los exponentes no altera la distribución de probabilidad final. Se definió la constante como el valor máximo actual: $M = \text{máx}_a(Q_t(a)/\tau)$. La ecuación computacional segura implementada es:

$$P(A_t = a) = \frac{e^{(Q_t(a)/\tau)-M}}{\sum_{b=1}^{k} e^{(Q_t(b)/\tau)-M}} \tag{1.8}$$

De esta forma, el mayor exponente a calcular siempre es exactamente $e^0 = 1$, haciendo matemáticamente imposible el desbordamiento sin sacrificar la precisión del algoritmo.

1.4. Evaluación, Configuración Experimental y Metodología

1.4.1. Parámetros de la Simulación

Para asegurar la validez estadística, reducir el ruido estocástico y permitir la reproducibilidad de este estudio, la experimentación se estandarizó bajo los siguientes parámetros metodológicos:

- **Horizonte Temporal (T):** Se evaluaron escenarios con $T = 1000$ pasos temporales para analizar el comportamiento asintótico, y escenarios reducidos de $T = 100/200$ pasos para visualizar la capacidad de reacción temprana (arranque en frío o *cold start*).
- **Complejidad del entorno (k):** Se fijó una cardinalidad de $\mathcal{A} = 10$ brazos concurrentes para los test comparativos finales.
- **Consistencia Estadística:** Todos los resultados numéricos y gráficos extraídos en este informe representan la media aritmética de 500 ejecuciones independientes (*runs*). Se utilizó la semilla pseudoaleatoria fija `SEED = 442` en las librerías NumPy y Python base.

1.4.2. Métricas de Rendimiento

La evaluación se basó en el registro histórico de tres métricas en cada instante t:

1. **Porcentaje de Selección Óptima:** Proporción de *runs* en las que el agente logró identificar y accionar el brazo con la mayor esperanza matemática real a^*.
2. **Recompensa Promedio (*Average Reward*):** La ganancia media empírica obtenida en el instante temporal exacto t a través de todas las simulaciones.
3. **Regret Acumulado:** La integral discreta del coste de oportunidad sufrido por el agente hasta el instante t.

1.5. Análisis de Resultados

1.5.1. Dinámica Exploratoria de ϵ-Greedy

La experimentación sobre la familia ϵ-greedy (Figura 1.1) confirma su comportamiento característico. El caso extremo de explotación pura ($\epsilon = 0$, asumiendo la fase de inicialización) muestra un ascenso empinado inicial debido a la rápida explotación del mejor brazo encontrado en los primeros 10 pasos. Sin embargo, su porcentaje de acción óptima se estanca rápidamente por debajo del 50 %. Esto ocurre porque el mejor brazo de la inicialización no siempre es el óptimo global, y la falta total de exploración lo condena al fracaso en una de cada dos simulaciones.

Por el contrario, configuraciones moderadas como $\epsilon = 0{,}1$ logran un crecimiento sostenido, superando el 80 % de selección óptima de forma constante.

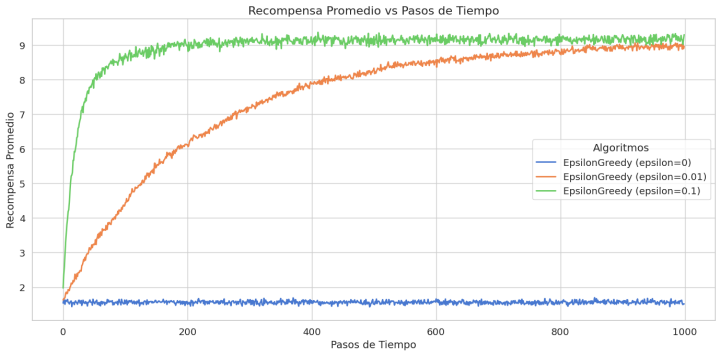

Figura 1.1: Evolución temporal de la recompensa promedio y ratio de acción óptima variando ϵ.

No obstante, la métrica global oculta una severa ineficiencia subyacente. El análisis estadístico de la distribución de selecciones (Figura 1.2) expone la debilidad teórica de este enfoque. ϵ-greedy reparte su probabilidad exploratoria (ϵ) de forma estrictamente plana. Empíricamente, esto significa que el algoritmo continuará visitando el peor brazo posible del entorno con exactamente la misma frecuencia que la segunda mejor opción, malgastando recursos computacionales y temporales vitales.

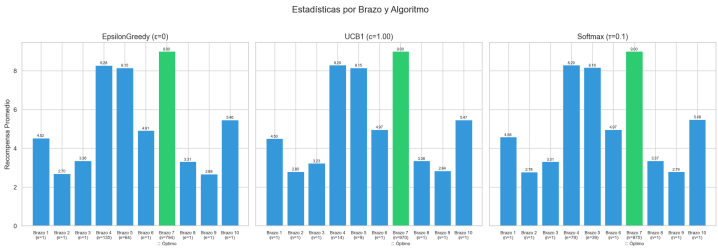

Figura 1.2: Histograma de frecuencias de selección por brazo.

1.5.2. El Fenómeno de la Escala y el Regret Asintótico

Una observación empírica fundamental del estudio cruzado fue la drástica diferencia en la magnitud absoluta del *regret* entre distribuciones. Tras una ejecución estandarizada, el *regret* final para el entorno **Bernoulli** resultó en aproximadamente **7.31 unidades**, mientras que para el **Binomial** ($n = 100$) escaló hasta **293.55 unidades**.

Esto no indica de ninguna manera que el algoritmo sea defectuoso en el entorno Binomial. Ocurre por la propia definición matemática del *regret* (basada en el *gap* $\Delta_a = \mu^* - \mu_{a_t}$). En un entorno Bernoulli discreto, equivocarse de brazo cuesta como máximo 1 punto de recompensa. En un entorno Binomial sobre un lote de $n = 100$, errar en la selección puede penalizar hasta 30 o 40 puntos por un solo fallo en la iteración t.

Si evaluamos el comportamiento asintótico general (Figura 1.3), se visualiza la gran división teórica. ϵ-greedy (en cualquier configuración estática) arrastra un *regret* de crecimiento estrictamente **lineal**. Su pendiente es directamente proporcional al parámetro ϵ, confirmando que su exploración ciega acarrea un coste constante infinito. Por el contrario, los algoritmos basados en optimismo como UCB1 detienen drásticamente la selección de brazos malos con el paso del tiempo. Su *regret* se curva de forma cóncava, adoptando un crecimiento logarítmico y demostrando que es un algoritmo consistente frente a la cota de Lai y Robbins.

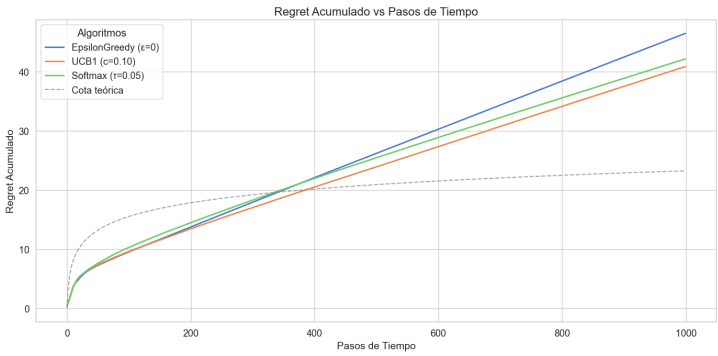

Figura 1.3: Evolución del Regret Acumulado.

1.5.3. Sensibilidad a la Distribución: UCB1 vs Softmax

El rendimiento relativo de la algoritmia avanzada invirtió sus tornas dependiendo de la "fuerza"de la señal estocástica, un resultado evidente en la comparativa de la Figura 1.4.

En el entorno Bernoulli: La recompensa aporta un solo bit de información (0 o 1). La altísima varianza relativa oscurece las diferencias reales entre esperanzas, engañando a los algoritmos. Aquí, **UCB1 resulta ser el ganador absoluto**. Al mantener registros de límites de confianza pesimistas/optimistas muy estrictos, supera la fase ruidosa inicial y converge inexorablemente. Softmax, por el contrario, sufre de sobre-exploración crónica al interpretar el ruido como variaciones de valor real, obligando a realizar sintonizaciones del hiperparámetro τ excesivamente complejas para un uso práctico.

En los entornos Binomial y Normal: En la distribución Binomial (evaluada con $n = 100$) y en la Normal, la "distancia"numérica entre la esperanza matemática del brazo óptimo y el resto es mucho más evidente desde las primeras iteraciones. En estos entornos, UCB1 (con c estándar de 1,0 o 0,5) y Softmax ($\tau = 1{,}0$) brillan excepcionalmente. La señal fuerte permite a Softmax discriminar casi de inmediato las opciones subóptimas y concentrar la probabilidad de selección en los dos o tres mejores brazos, logrando ratios de selección óptima superiores al 90 % en apenas una fracción del tiempo necesario en el entorno Bernoulli.

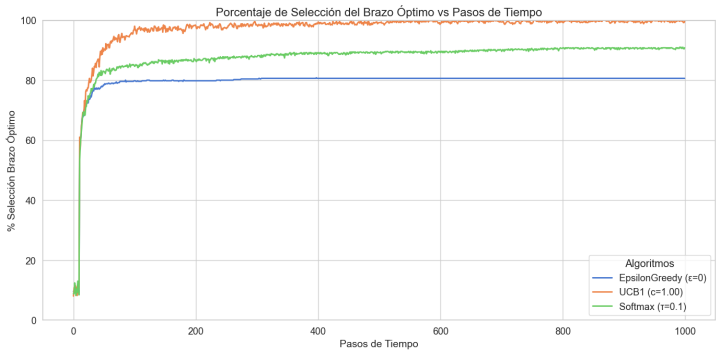

Figura 1.4: omparativa del porcentaje de selección del brazo óptimo.

1.6. Discusión y Conclusiones

El análisis riguroso de las métricas estadísticas obtenidas sobre la implementación de algoritmos básicos y avanzados, evaluados bajo la presión de diversas naturalezas estocásticas, permite consolidar las siguientes conclusiones definitivas:

- **La ineficacia matemática de la exploración ciega:** Se ha demostrado empíricamente que, aunque ϵ-greedy con un valor conservador (ej. $\epsilon = 0{,}1$) es una heurística fácil de implementar y muy robusta en las primeras etapas de aprendizaje, arrastra un defecto arquitectónico grave. Su exploración probabilística uniforme causa que el agente acumule un *regret* asintótico lineal infinito, inhabilitándolo para sistemas autónomos que deban operar en horizontes temporales extremadamente largos, a menos que se implementen complejas políticas de decaimiento del parámetro.
- **Solidez Teórica de los Intervalos de Confianza (UCB):** UCB1 se posiciona como el paradigma más robusto y consistente de forma global. Ya sea operando bajo señales continuas claras (distribuciones Normales) o bajo señales discretas de alto ruido relativo (distribuciones Bernoulli), su mecanismo de penalización basado en la desigualdad de Hoeffding maneja la incertidumbre de forma brillante. Constituye la opción metodológica por defecto para problemas de multi-armed bandits debido a que requiere mínimo ajuste paramétrico de su factor c y empareja la cota teórica inferior logarítmica descrita en la teoría de juegos.
- **El talón de Aquiles de la exploración térmica (Softmax):** El algoritmo basado en la distribución de Boltzmann presenta un desempeño excepcional en escenarios de baja varianza o "señal fuerte", como la distribución Binomial, logrando converger más rápido que sus contrapartes deterministas. No obstante, su dependencia absoluta del hiperparámetro de temperatura τ limita severamente su automatización. Mientras que UCB1 se ajusta automáticamente mediante el contador de visitas, en Softmax, un cambio en la magnitud escalar del entorno (de recompensas en rango $[0, 1]$ a rango $[0, 100]$) demanda obligatoriamente un reajuste manual completo del hiperparámetro para evitar la degradación a políticas puramente *greedy* o aleatorias.
- **La interpretación relativa del Regret:** El estudio confirma que la métrica del *regret* acumulado no es un valor absoluto, sino estrictamente relativo a la separación real de las esperanzas matemáticas del dominio particular. Obtener *regret* de cientos de puntos en distribuciones Binomiales no es indicativo de un fallo algorítmico, sino del coste inherente a operar en escalas mayores. Al diseñar sistemas de aprendizaje por refuerzo, los ingenieros deben contextualizar las métricas de penalización de acuerdo con la varianza máxima y el *gap* de las distribuciones subyacentes.

Capítulo 2

Aprendizaje en Entornos Complejos

2.1. Introducción

- **Descripción del problema:** Este segundo capitulo aborda dos escenarios complementarios: (1) la navegación en un entorno cuadriculado discreto, *FrozenLake-v1*, donde un agente debe encontrar una ruta óptima hacia un objetivo evitando agujeros [6]; y (2) el control de un péndulo invertido en *CartPole-v1*, con un espacio de estados continuo de 4 dimensiones donde los métodos tabulares resultan inviables y se requieren aproximadores de funciones [6, 1].
- **Motivación:** Comprender el comportamiento de los métodos clásicos de Aprendizaje por Refuerzo (RL) es vital antes de abordar entornos continuos complejos. Los algoritmos tabulares de Monte Carlo ofrecen una base matemática sólida para evaluar cómo un agente aprende de la experiencia empírica mediante retornos episódicos completos. Por su parte, los métodos de Diferencias Temporales (TD) aportan un enfoque complementario fundamental al permitir aprender paso a paso (*bootstrapping*) actualizando estimaciones sin tener que esperar al final del episodio. Finalmente, los métodos aproximados —lineales y no lineales— permiten generalizar a espacios de estados continuos, ilustrando fenómenos como la *deadly triad* y las técnicas de estabilización que la mitigan [1].
- **Objetivos:** Implementar, evaluar y comparar: (1) algoritmos tabulares en FrozenLake-v1, incluyendo tres variantes de Monte Carlo (*On-Policy* con y sin decaimiento de exploración, *Off-Policy* con Muestreo por Importancia Ponderado) y dos métodos de Diferencias Temporales (*SARSA On-Policy* y *Q-Learning Off-Policy*); y (2) *SARSA Semi-Gradiente* (aproximación lineal, on-policy) y *Deep Q-Learning* (DQN, red neuronal, off-policy) en CartPole-v1, analizando convergencia, estabilidad y generalización en todos los escenarios.
- **Organización:** La Sección 2 define formalmente el problema y el contexto teórico para ambos entornos; la Sección 3 detalla el pseudocódigo y la justificación de todos los algoritmos empleados (tabulares —Monte Carlo y TD— y aproximados); la Sección 4 expone la configuración experimental y analiza críticamente los resultados obtenidos en ambos escenarios; finalmente, la Sección 5 resume los hallazgos empíricos.

2.2. Desarrollo

El problema de toma de decisiones se modela formalmente como un Proceso de Decisión de Markov (MDP). Un MDP está definido por una tupla $\langle S, A, P, R, \gamma \rangle$, donde S es el conjunto de estados, A el conjunto de acciones, P la probabilidad de transición, R la función de recompensa y $\gamma \in [0, 1]$ el factor de descuento.

2.2.1. Entorno discreto: FrozenLake-v1

En *FrozenLake-v1*, el espacio de estados S consta de 16 celdas discretas dispuestas en una cuadrícula de 4×4, y el espacio de acciones A permite 4 movimientos discretos (Izquierda, Abajo, Derecha, Arriba). El agente recibe una recompensa $R = +1$ si alcanza el objetivo, y $R = 0$ en cualquier otro caso o si cae en un agujero. Al tratarse de un espacio de estados finito, es posible almacenar un valor $Q(s, a)$ por cada par estado-acción en una tabla.

2.2.2. Entorno continuo: CartPole-v1

En *CartPole-v1*, el espacio de estados S es continuo y de 4 dimensiones (posición del carro, velocidad del carro, ángulo del poste y velocidad angular), mientras que A es discreto con 2 acciones (empujar a izquierda o derecha). La recompensa es $R = +1$ por cada paso en que el poste se mantiene en equilibrio; el episodio termina si el poste supera 12° o el carro sale de los límites, o tras 500 pasos (truncamiento por éxito). A diferencia de FrozenLake, el espacio continuo de CartPole exige modelos parametrizados que aproximen $Q(s, a)$ y generalicen entre estados cercanos.

2.2.3. Trabajos Relacionados y Contexto

En entornos discretos con recompensas dispersas (donde la señal de aprendizaje es binaria y suele obtenerse únicamente al alcanzar un objetivo final), es viable utilizar métodos tabulares que almacenan el valor exacto de cada par estado-acción. Por el contrario, en espacios de estados continuos con recompensas densas (evaluadas en cada paso temporal), la dimensionalidad del entorno hace inviable el enfoque tabular, forzando el uso de aproximadores de funciones paramétricos.

Para el caso de espacios discretos, este trabajo analiza dos enfoques tabulares clásicos. Primero, los métodos de Monte Carlo (MC), que basan su aprendizaje en promediar retornos empíricos completos, requiriendo que los episodios terminen para poder propagar la recompensa obtenida a la función $Q(s, a)$. Como contraparte teórica, los métodos de Diferencias Temporales (TD) permiten aprender paso a paso (*bootstrapping*), actualizando las estimaciones presentes a partir de las estimaciones del futuro inmediato. Dentro de TD, se contras-

ta SARSA (un enfoque *on-policy* que genera trayectorias más conservadoras al integrar la exploración real del agente) frente a Q-Learning (un enfoque *off-policy* optimista que asume futuras decisiones perfectas mediante el operador máx).

Para abordar los entornos con espacios de estados continuos, Sutton y Barto [1] formalizan los métodos TD con aproximación de funciones, como SARSA Semi-Gradiente, que utiliza combinaciones lineales de características. Asimismo, para mitigar las inestabilidades teóricas que surgen al aplicar aproximadores no lineales (la conocida *deadly triad*), Mnih et al. [5] introdujeron la arquitectura DQN, que logra estabilizar el entrenamiento en espacios complejos mediante buffers de repetición de experiencia (*Experience Replay*) y redes objetivo (*Target Networks*).

Para evaluar rigurosamente el rendimiento y la velocidad de convergencia de estos algoritmos durante su entrenamiento, es fundamental monitorizar métricas que reflejen el proceso de optimización del agente. Por ello, la experimentación de este documento se apoyará en representaciones gráficas basadas en dos indicadores clave: la evolución de la recompensa media y la longitud de los episodios a lo largo del tiempo. Mientras que la recompensa promedio muestra directamente la tasa de éxito y la capacidad para maximizar el retorno, la longitud del episodio actúa como un indicador temprano fundamental de aprendizaje, evidenciando de forma visual el punto exacto en el que el agente abandona la deambulación aleatoria inicial y comienza a optimizar sus rutas hacia los estados terminales.

2.3. Algoritmos

A continuación, se describen los algoritmos implementados, detallando su funcionamiento.

2.3.1. Métodos Tabulares de Monte Carlo

A continuación se detallan los algoritmos tabulares implementados para FrozenLake-v1. Para la estimación de la función de valor, se ha utilizado el enfoque de *Todas las Visitas* (*Every-Visit*), que actualiza la estimación del par estado-acción en cada ocurrencia dentro de un episodio.

Monte Carlo On-Policy (con decaimiento)

Este método aprende la función de valor de la misma política que utiliza para tomar decisiones. Para garantizar la exploración de nuevos estados y evitar óptimos locales, emplea una política ϵ-greedy. Se utiliza una media incremental para actualizar $Q(s,a)$ sin necesidad de almacenar todo el historial de retornos, como se ilustra en el **Algorithm 1**. Para asegurar que el agente termine explotando la política óptima, el parámetro ϵ se reduce tras cada episodio.

Algorithm 1 Monte Carlo On-Policy (Every-Visit) con decaimiento de ϵ

Require: Entorno episódico, $\gamma \in (0,1]$, $\epsilon > 0$, tasa de decaimiento
Ensure: Estimación de la política óptima $Q \approx q_*$

```
Inicializar Q(s,a) ← 0 y N(s,a) ← 0 para todo s ∈ S, a ∈ A
for cada episodio do
    Generar un episodio siguiendo la política ε-greedy basada en Q
    G ← 0
    for cada paso t del episodio en orden inverso do
        Obtener estado S_t, acción A_t y recompensa R_{t+1}
        G ← γG + R_{t+1}
        N(S_t, A_t) ← N(S_t, A_t) + 1
        Q(S_t, A_t) ← Q(S_t, A_t) + 1/N(S_t, A_t) [G − Q(S_t, A_t)]
    end for
    ε ← máx(ε × decaimiento, ε_min)        ▷ Transición a la explotación
end for
```

Monte Carlo On-Policy (ϵ-fijo)

Como caso de control (o *baseline*) para contrastar el efecto del decaimiento, se implementa una variante donde la probabilidad de exploración se mantiene constante. Esto significa que el agente nunca deja de explorar aleatoriamente, lo que impide que la política converja a una estrategia puramente determinista y óptima, tal y como se formaliza en el **Algorithm 2**.

Algorithm 2 Monte Carlo On-Policy con ϵ-fijo (Sin decaimiento)

Require: Entorno episódico, $\gamma \in (0,1]$, $\epsilon > 0$ constante
Ensure: Estimación de la política Q (subóptima por exploración perpetua)

```
Inicializar Q(s,a) ← 0 y N(s,a) ← 0 para todo s ∈ S, a ∈ A
for cada episodio do
    Generar un episodio siguiendo la política ε-greedy basada en Q
    G ← 0
    for cada paso t del episodio en orden inverso do
        Obtener estado S_t, acción A_t y recompensa R_{t+1}
        G ← γG + R_{t+1}
        N(S_t, A_t) ← N(S_t, A_t) + 1
        Q(S_t, A_t) ← Q(S_t, A_t) + 1/N(S_t, A_t) [G − Q(S_t, A_t)]
    end for
            ▷ No se actualiza ε; se mantiene la exploración constante
end for
```

Monte Carlo Off-Policy

Aprende una política objetivo óptima π (estrictamente *greedy*) mientras sigue una política de comportamiento b exploratoria (ϵ-greedy). Para corregir la diferencia entre las distribuciones de probabilidad, emplea el Muestreo por Importancia Ponderado (ver **Algorithm 3**).

Algorithm 3 Monte Carlo Off-Policy (Importance Sampling Ponderado)

Require: Entorno episódico, $\gamma \in (0, 1]$
Ensure: Estimación de la política óptima $Q \approx q_*$

```
Inicializar Q(s,a) ← 0 y C(s,a) ← 0 para todo s ∈ S, a ∈ A
for cada episodio do
    Generar un episodio usando política de comportamiento b (ε-greedy)
    G ← 0, W ← 1
    for cada paso t del episodio en orden inverso do
        Obtener S_t, A_t y R_{t+1}
        G ← γG + R_{t+1}
        C(S_t, A_t) ← C(S_t, A_t) + W
        Q(S_t, A_t) ← Q(S_t, A_t) + (W / C(S_t, A_t)) [G − Q(S_t, A_t)]
        if A_t ≠ arg máx_a Q(S_t, a) then
            break  ▷ La acción no coincide con la política objetivo π
        end if
        W ← W × 1 / b(A_t | S_t)
    end for
end for
```

2.3.2. Métodos Tabulares de Diferencias Temporales (TD)

A diferencia de Monte Carlo, los métodos de Diferencias Temporales (TD) actualizan sus estimaciones paso a paso, utilizando el método de *bootstrapping* (aprender una estimación a partir de otra estimación actual) sin necesidad de esperar a que finalice el episodio. Para la resolución del entorno discreto, se han implementado los algoritmos SARSA y Q-Learning. Ambos utilizan una política ϵ-greedy constante (sin decaimiento) para mantener activa la exploración.

SARSA (On-Policy)

SARSA (*State-Action-Reward-State-Action*) es un método **on-policy** que evalúa y mejora la misma política exploratoria que el agente utiliza para interactuar con el entorno. La regla de actualización depende directamente de la acción A' que el agente haya decidido tomar (la cual incluye exploración) en el siguiente estado S', tal y como se detalla en el **Algorithm 4**. Al sufrir penalizaciones por sus propias acciones aleatorias cerca de los obstáculos, tiende a converger hacia caminos más conservadores.

Q-Learning (Off-Policy)

Q-Learning es un método **off-policy** que busca aproximar la política óptima independientemente de la política de comportamiento que esté dictando las acciones del agente (ϵ-greedy). Para lograrlo, su regla de actualización (ver **Algorithm 5**) utiliza el operador máx sobre los valores del siguiente estado

Algorithm 4 SARSA (On-Policy TD Control)

Require: Entorno episódico, $\alpha \in (0, 1]$, $\gamma \in (0, 1]$, $\epsilon > 0$
Ensure: Estimación de la función de valor-acción Q

```
Inicializar Q(s,a) ← 0 para todo s ∈ S, a ∈ A
for cada episodio do
    Inicializar S
    Elegir A desde S usando política derivada de Q (ε-greedy)
    while S no sea terminal do
        Ejecutar acción A, observar recompensa R y siguiente estado S'
        Elegir A' desde S' usando política derivada de Q (ε-greedy)
        Q(S,A) ← Q(S,A) + α [R + γQ(S',A') − Q(S,A)]
        S ← S', A ← A'
    end while
end for
```

S', asumiendo que el agente tomará en el futuro la decisión óptima. Esto lo hace un algoritmo más optimista y rápido en hallar la ruta más corta.

Algorithm 5 Q-Learning (Off-Policy TD Control)

Require: Entorno episódico, $\alpha \in (0, 1]$, $\gamma \in (0, 1]$, $\epsilon > 0$
Ensure: Estimación de la función de valor-acción óptima $Q \approx q_*$

```
Inicializar Q(s,a) ← 0 para todo s ∈ S, a ∈ A
for cada episodio do
    Inicializar S
    while S no sea terminal do
        Elegir A desde S usando política derivada de Q (ε-greedy)
        Ejecutar acción A, observar recompensa R y siguiente estado S'
        Q(S,A) ← Q(S,A) + α [R + γ máx_a Q(S',a) − Q(S,A)]
        S ← S'
    end while
end for
```

2.3.3. Métodos con Aproximación de Funciones

Cuando el espacio de estados es continuo, como en CartPole-v1, los métodos tabulares son inviables. Los *métodos aproximados* reemplazan la tabla $Q(s, a)$ por un modelo parametrizado que generaliza entre estados similares. Ambos algoritmos siguientes emplean política ε-greedy con decaimiento multiplicativo por episodio.

SARSA Semi-Gradiente con aproximación lineal

Método **on-policy** que aprende la función de valor de la misma política que utiliza para tomar decisiones. Utiliza un vector de pesos $\mathbf{w}$ y una función de características $\mathbf{x}(s, a)$ para aproximar $\hat{Q}(s, a, \mathbf{w}) = \mathbf{w}^\top \mathbf{x}(s, a)$. La codificación estado-acción normaliza las 4 variables por sus rangos típicos (scales = [4,8, 3,0, 0,418, 3,0]) y genera un vector de dimensión $4 \times 2 = 8$, donde el estado normalizado se coloca en la posición correspondiente a la acción seleccionada y el resto se rellena con ceros. La actualización semi-gradiente, donde el

gradiente no se propaga a través del target, es:

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha \cdot \delta \cdot \mathbf{x}(s,a), \quad \delta = R + \gamma\, \hat{Q}(s', a', \mathbf{w}) - \hat{Q}(s, a, \mathbf{w}) \tag{2.1}$$

Algorithm 6 SARSA Semi-Gradiente con aproximación lineal

Require: Entorno, γ, α, ε, decaimiento
Inicializar $\mathbf{w}$ aleatorio
for cada episodio **do**
  $s \leftarrow$ estado inicial; $a \leftarrow \varepsilon$-greedy$(\hat{Q})$
  while no terminal **do**
    Ejecutar a, observar r, s'
    $a' \leftarrow \varepsilon$-greedy$(\hat{Q}(s', \cdot))$ si no terminal
    $\delta \leftarrow r + \gamma \hat{Q}(s', a', \mathbf{w}) - \hat{Q}(s, a, \mathbf{w})$
    $\mathbf{w} \leftarrow \mathbf{w} + \alpha\delta\mathbf{x}(s,a)$; $s \leftarrow s'$; $a \leftarrow a'$
  end while
  Decaer ε
end for

Deep Q-Learning (DQN)

Algoritmo **off-policy** que emplea una red neuronal profunda para aproximar $Q(s,a)$. La red es un perceptrón multicapa (MLP): el estado continuo de 4 dimensiones alimenta dos capas ocultas de 64 neuronas con activación ReLU, y la capa de salida produce un valor Q por cada acción (2), totalizando $\sim$4 290 parámetros. DQN incorpora dos mecanismos contra la *deadly triad*:

- **Experience Replay:** Buffer circular que almacena experiencias $(s, a, r, s', \text{done})$ y muestrea mini-batches aleatorios, rompiendo la correlación temporal y reutilizando datos.
- **Target Network:** Copia de la red principal sincronizada cada T pasos, que fija temporalmente los valores objetivo y evita ciclos de retroalimentación.

La red se entrena minimizando el error cuadrático medio (MSE) con el optimizador Adam.

Algorithm 7 Deep Q-Learning (DQN)

Require: Entorno, γ, α, ε, decaimiento, buffer B, batch size, freq. sync. T
Inicializar Q_θ, Q_{θ^-} iguales; buffer $\mathcal{D}$
for cada episodio **do**
  $s \leftarrow$ estado inicial
  while no terminal **do**
    $a \leftarrow \varepsilon$-greedy(Q_θ); ejecutar a; almacenar (s, a, r, s', d) en $\mathcal{D}$
    if $|\mathcal{D}| \geq$ batch_size **then**
      $y_i \leftarrow r_i + (1 - d_i)\gamma \max_{a'} Q_{\theta^-}(s'_i, a')$
      Minimizar $\sum (Q_\theta(s_i, a_i) - y_i)^2$
    end if
    Cada T pasos: $\theta^- \leftarrow \theta$; $s \leftarrow s'$
  end while
  Decaer ε
end for

2.4. Evaluación y Experimentos

2.4.1. Métodos Tabulares en FrozenLake-v1

Configuración Experimental

Para evaluar la robustez y escalabilidad de los métodos tabulares implementados, los experimentos se han ejecutado en dos variantes del entorno `FrozenLake-v1`: la cuadrícula clásica de 4×4 y una versión extendida de 8×8. Ambos entornos se inicializaron en modo determinista (`is_slippery=False`) para aislar el rendimiento del algoritmo de la estocasticidad del entorno.

Se realizaron baterías de pruebas extensas (entre 5,000 y 100,000 episodios dependiendo de la convergencia del algoritmo). Dada la dimensionalidad del entorno 8×8, se ha utilizado una ventana de media móvil para suavizar el ruido inherente a la exploración y apreciar claramente las tendencias de aprendizaje.

Resultados de Monte Carlo en el Entorno 4×4

En el entorno de menor dimensionalidad, el horizonte temporal para alcanzar la meta es corto, lo que facilita la propagación de las recompensas en los métodos de Monte Carlo.

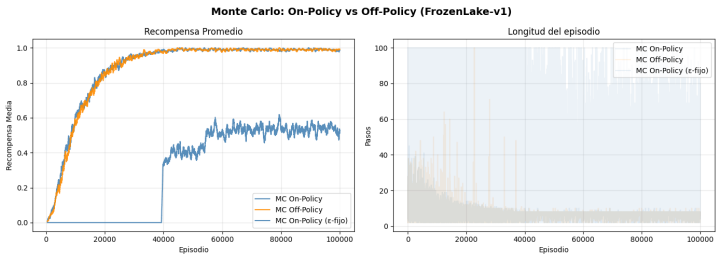

Figura 2.1: Recompensa promedio y longitud de episodios para el entorno FrozenLake 4×4 usando Monte Carlo (100,000 episodios).

Al analizar la Figura 2.1, se destacan los siguientes comportamientos:

- **MC On-Policy (con decaimiento):** Muestra una convergencia extremadamente rápida. En los primeros miles de episodios alcanza una recompensa promedio de 1,0 (100 % de éxito) y reduce la longitud de los episodios a unos 6 pasos, que es la ruta óptima.
- **MC Off-Policy:** Alcanza también la convergencia total, pero su curva de aprendizaje es ligeramente más gradual debido a que el Muestreo por Importancia interrumpe las actualizaciones cuando las acciones no coinciden con la política *greedy*.
- **MC On-Policy (ϵ-fijo):** Al mantener una tasa de exploración constante, el agente nunca logra una política determinista. Su recompensa oscila entre 0,6 y 0,8, evidenciando los pasos en falso provocados por la exploración forzada.

Resultados de Monte Carlo en el Entorno 8×8

El entorno 8×8 representa un desafío significativamente mayor. La cantidad de estados se cuadruplica (64 estados) y la longitud de la ruta óptima crece.

El análisis revela diferencias críticas:

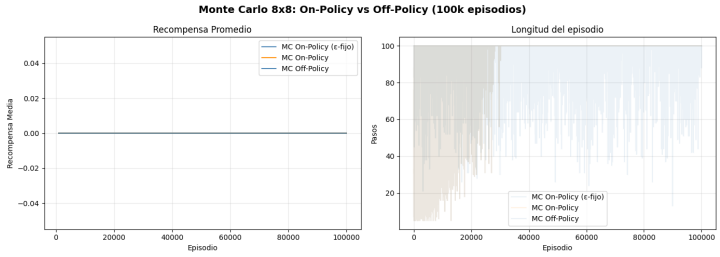

Figura 2.2: Recompensa promedio y longitud de episodios para el entorno FrozenLake 8×8 usando Monte Carlo (100,000 episodios).

- **MC On-Policy (con decaimiento):** Vuelve a ser el más robusto. Tras una fase de exploración de ∼60,000 episodios, escala rápidamente hasta el éxito total y minimiza la longitud de la ruta.
- **MC Off-Policy:** Sufre severamente. Al ser trayectorias largas, la probabilidad de que la política de comportamiento ejecute una secuencia idéntica a la objetivo es mínima, anulando las actualizaciones.
- **MC On-Policy (ϵ-fijo):** Colapsa absolutamente. Un 20 % de acciones aleatorias en un mapa grande garantiza la caída del agente, manteniendo la recompensa en 0.

Resultados de Diferencias Temporales en el Entorno 4×4

A diferencia de Monte Carlo, los métodos TD (SARSA y Q-Learning) actualizan la tabla $Q(s, a)$ en cada paso, lo que acelera notablemente la propagación de la recompensa sin necesidad de esperar a que el episodio termine.

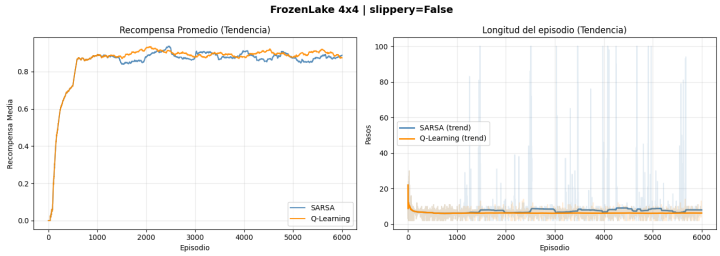

Figura 2.3: Recompensa promedio y longitud de episodios para el entorno FrozenLake 4×4 usando SARSA y Q-Learning.

La Figura 2.3 evidencia la superioridad en eficiencia de aprendizaje paso a paso:

- **Q-Learning (Off-Policy):** Muestra la propagación de valores hacia atrás más agresiva. Gracias a la actualización mediante el operador máx, asume siempre el camino óptimo futuro. Esto se refleja en una caída muy abrupta y temprana en la gráfica de longitud de episodios, encontrando la ruta mínima rápidamente.
- **SARSA (On-Policy):** Su convergencia es ligeramente más conservadora. Al incluir empíricamente las caídas provocadas por la política ϵ-greedy real, penaliza más los estados adyacentes a los agujeros, tardando un poco más en estabilizar la longitud de sus episodios hacia el óptimo absoluto.

Resultados de Diferencias Temporales en el Entorno 8×8

La transición al mapa de 64 estados pone a prueba la escalabilidad de los métodos de *bootstrapping* frente a la escasez de recompensas.

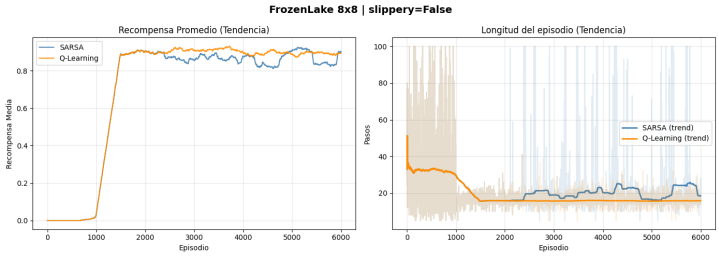

Figura 2.4: Recompensa promedio y longitud de episodios para el entorno FrozenLake 8×8 usando SARSA y Q-Learning.

Observando la Figura 2.4, extraemos las siguientes conclusiones sobre la escalabilidad:

- **Meseta Inicial (Exploración Combinatoria):** Ambos algoritmos sufren un claro estancamiento inicial donde la recompensa es nula y la longitud del episodio es errática. Esto representa empíricamente la "maldición de la dimensionalidad": el agente deambula al azar hasta que, por pura probabilidad, tropieza con la meta ($R = +1$) por primera vez.
- **Estabilización:** Una vez descubierta la meta, Q-Learning vuelve a demostrar una estabilización más veloz de la ruta óptima. Pese a mantener un ϵ constante, ambos algoritmos TD logran resolver el entorno (a diferencia de MC con ϵ-fijo), ya que sus penalizaciones locales paso a paso les impiden repetir ciclos suicidas infinitos.

Análisis Visual de las Políticas Aprendidas en 8×8

Para comprender profundamente el impacto práctico de la naturaleza *on-policy* frente a la *off-policy*, resulta fundamental visualizar el camino final que cada agente decide trazar sobre la cuadrícula.

Al contrastar la Figura 2.5 y la Figura 2.6, se manifiesta claramente el comportamiento diferencial descrito en la teoría:

- **La ruta de Q-Learning:** El agente traza el camino más corto posible, bordeando peligrosamente los agujeros. Al utilizar el operador máx en su regla de actualización, ignora completamente el riesgo introducido por la política de exploración ϵ-greedy que lo rige. Q-Learning asume que en el futuro no cometerá errores ni tomará decisiones aleatorias, por lo que no guarda precaución por los precipicios cercanos.
- **La ruta de SARSA (Conservadora):** El agente opta por un camino más largo pero mucho más seguro, alejándose intencionadamente de los agujeros cuando le es posible. Dado que SARSA actualiza sus valores basándose en la acción real A' que tomará en el futuro (la cual incluye el ϵ de exploración aleatoria), el agente sabe que

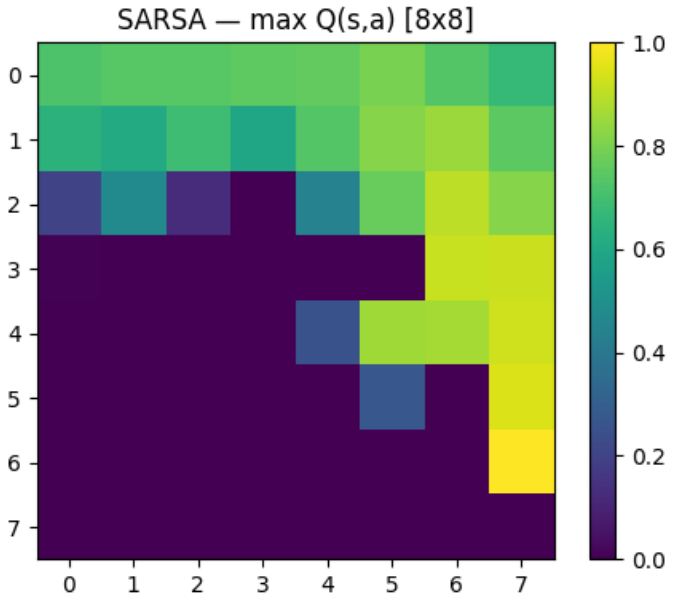

Figura 2.5: Política de SARSA.

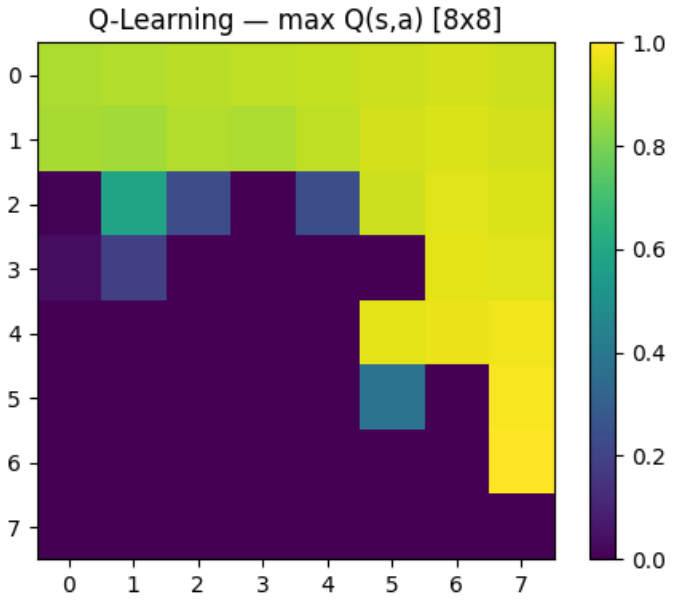

Figura 2.6: Política de Q-Learning.

caminar pegado a un agujero conlleva una probabilidad de caer debido a su propia aleatoriedad. Por consiguiente, penaliza los estados contiguos al peligro y prefiere rodearlos.

2.4.2. Métodos Aproximados en CartPole-v1

Configuración Experimental

Ambos algoritmos se ejecutaron en el entorno `CartPole-v1` de Gymnasium durante 5,000 episodios con semilla fija seed = 2026 para garantizar reproducibilidad. Se registraron las recompensas y la longitud de cada episodio, y se evaluó la política greedy pura (sin exploración) con los mejores pesos encontrados durante el entrenamiento. Nótese que SARSA usa $\alpha = 0{,}01$ (mayor que DQN), habitual en modelos lineales, y el decaimiento de ε se aplica **por episodio** en ambos algoritmos.

Parámetro	SARSA Semi-Grad.	DQN
α	0.01	0.001
γ	0.99	0.99
$\varepsilon_0/\varepsilon_{\text{mín}}$/decay	1.0 / 0.01 / 0.9995	1.0 / 0.01 / 0.9995
Batch size / Buffer	—	64 / 10,000
Target sync / Optim.	—	100 pasos / Adam

Tabla 2.1: Hiperparámetros de los algoritmos aproximados (5,000 episodios, seed=2026).

Resultados de SARSA Semi-Gradiente

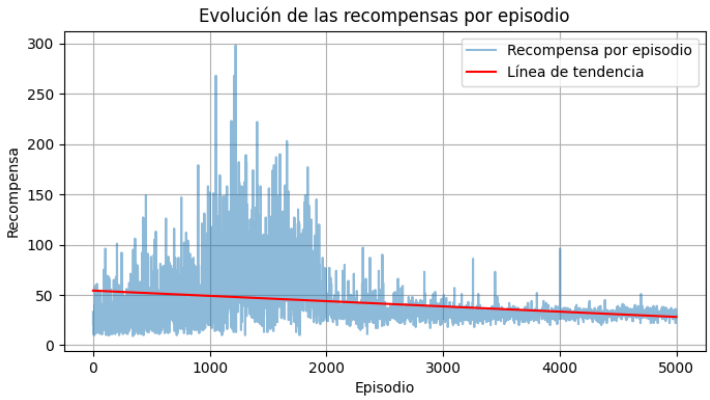

Figura 2.7: Recompensas por episodio — SARSA Semi-Gradiente.

La Figura 2.7 muestra tres fases diferenciadas: (1) **aprendizaje** (ep. 0–1,500), donde el agente mejora progresivamente alcanzando un pico de **298 pasos** (ep. 1,222) —la linealidad local $\sin\theta \approx \theta$ favorece al modelo de solo 8 parámetros—; (2) **declive** (ep. 1,500–5,000), donde la *deadly triad* provoca que las actualizaciones no converjan, arrastrando los pesos hacia configuraciones subóptimas; y (3) **estabilización final** en ∼20–30 pasos, equivalente a una política casi aleatoria. En la evaluación greedy (con los mejores pesos del ep. 1,222), el agente mantiene **298 pasos**.

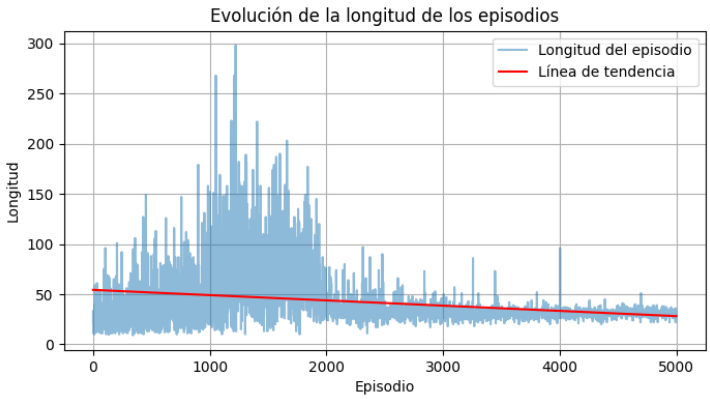

Figura 2.8: Longitud de episodios — SARSA Semi-Gradiente.

La Figura 2.8 confirma el mismo patrón descendente. La cau-

sa principal es la ausencia de mecanismos de estabilización: cada actualización modifica directamente los pesos sin buffer ni red objetivo, amplificando la varianza. Además, con solo 8 parámetros, el modelo no puede representar la función Q real, que involucra términos como $\sin(\theta)$, $\cos(\theta)$ y productos cruzados.

Resultados de DQN

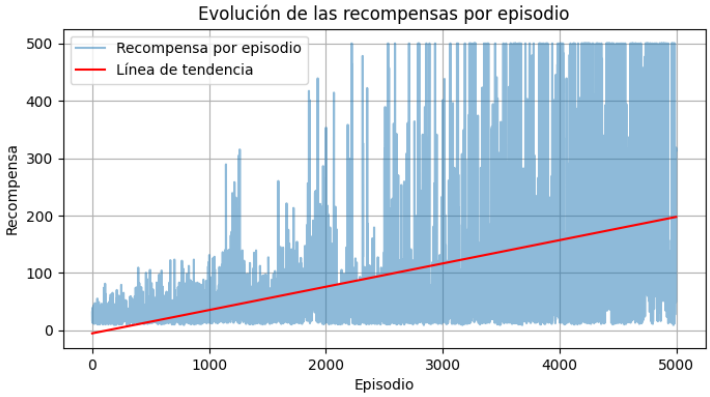

Figura 2.9: Recompensas por episodio — DQN.

La Figura 2.9 revela una dinámica opuesta a SARSA: (1) **exploración lenta** (ep. 0–2,000) con $\varepsilon > 0{,}37$, donde el buffer acumula experiencias variadas; (2) **primer episodio perfecto** de **500 pasos** en el ep. 2,222, demostrando que la red ha capturado la dinámica del entorno; (3) **consolidación** (ep. 2,000–4,000) con recompensas crecientes de forma sostenida; y (4) **fase madura** (ep. 4,000–5,000), con media ∼194 y **220 episodios** alcanzando la recompensa máxima de 500.

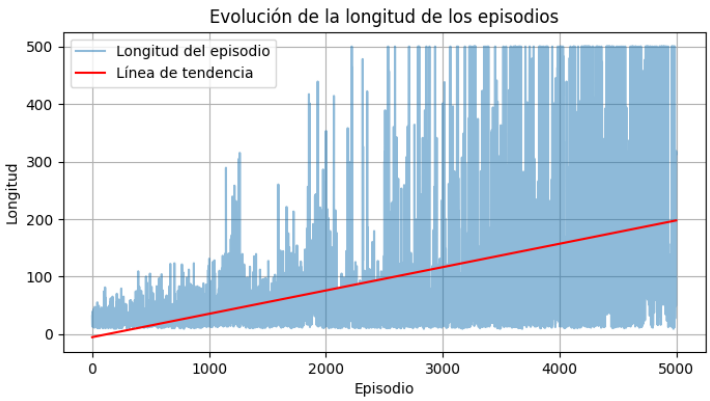

Figura 2.10: Longitud de episodios — DQN.

La Figura 2.10 confirma la tendencia creciente sostenida, demostrando que DQN **aprende sin degradarse**. El Experience Replay rompe la correlación temporal y la Target Network fija los valores objetivo, evitando la divergencia. Las caídas puntuales se explican por el ε residual (∼0.082 al final, sin alcanzar el mínimo de 0.01).

En evaluación greedy (con los mejores pesos del ep. 2,222), el agente logra **133 pasos**, significativamente menos que los 500 del entrenamiento. Esta brecha se debe a que el modelo alcanzó 500 con $\varepsilon \approx 0{,}33$ (un tercio de acciones aleatorias corregían decisiones subóptimas), y al eliminar la exploración, la política determinista queda expuesta a regiones donde la aproximación Q no es precisa.

Comparación entre algoritmos aproximados

Aspecto	SARSA S-G	DQN
Modelo	Lineal (8 params)	MLP (∼4,290)
Política	On-policy	Off-policy
Mejor recomp.	298 (59.6 %)	**500 (100 %)**
Tendencia	Decreciente	**Creciente**
Eval. greedy	**298**	133

Tabla 2.2: Comparación SARSA Semi-Gradiente vs. DQN.

Aunque DQN supera ampliamente a SARSA en recompensa máxima (500 vs. 298) y tendencia de aprendizaje (creciente vs. decreciente), SARSA obtiene mejor resultado en evaluación greedy (298 vs. 133). Esto sugiere que el modelo lineal, aunque limitado en capacidad, generaliza mejor su política al modo determinista, mientras que DQN puede sobreajustarse a las trayectorias generadas con exploración.

2.5. Conclusiones

- **Resultados principales (métodos tabulares):** Se ha demostrado empíricamente que, en entornos discretos (*FrozenLake-v1*), los métodos de Diferencias Temporales (TD) superan en eficiencia de convergencia a Monte Carlo gracias al *bootstrapping* (actualización paso a paso). Específicamente, Q-Learning (*off-policy*) encuentra la ruta óptima de forma más rápida y agresiva mediante el operador máx, mientras que SARSA (*on-policy*) traza rutas más conservadoras al tener en cuenta el riesgo real de su propia política exploratoria. Por su parte, dentro de la familia Monte Carlo, la variante *On-Policy* con decaimiento de exploración (ϵ) demostró ser la única viable computacionalmente frente a mapas de mayor tamaño.
- **Resultados principales (métodos aproximados):** DQN con *Experience Replay* y *Target Network* alcanza la recompensa máxima (500 pasos) en CartPole-v1 gracias a sus ∼4 290 parámetros y mecanismos de estabilización que mitigan la *deadly triad* [5, 1]. SARSA Semi-Gradiente, con solo 8 parámetros lineales, captura parcialmente la dinámica en la zona lineal pero exhibe una tendencia descendente final por la divergencia intrínseca del aprendizaje.
- **Limitaciones:** Los métodos de Monte Carlo adolecen de un problema estructural grave: las estimaciones solo pueden actualizarse al alcanzar un estado terminal, exhibiendo el enfoque *Off-Policy* severos problemas de varianza por el Muestreo por Importancia. Aunque TD corrige este retraso, sigue sufriendo fuertemente la "maldición de la dimensionalidad", evidenciada en la larga meseta inicial del entorno 8×8 provocada por la escasez de recompensas. En los métodos aproximados, buenas recompensas en entrenamiento no garantizan una política *greedy* robusta (DQN: 500→133; SARSA: 298→298). Con $\varepsilon_{\text{decay}} = 0{,}9995$ y 5 000 episodios, el ε final $\approx 0{,}082$ no alcanzó una fase de explotación profunda. Además,

la normalización de características resultó imprescindible para que el modelo lineal funcionara.

Bibliografía

[1] Sutton, R. S., & Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (2nd ed.). MIT Press.

[2] Auer, P., Cesa-Bianchi, N., & Fischer, P. (2002). *Finite-time Analysis of the Multiarmed Bandit Problem.* Machine Learning, 47(2), 235-256.

[3] Lai, T. L., & Robbins, H. (1985). *Asymptotically efficient adaptive allocation rules.* Advances in Applied Mathematics, 6(1), 4-22.

[4] Luis Daniel Hernández. (2025). *Código fuente y experimentación: Problema del Bandido de k-brazos.* GitHub. Recuperado de: `https://github.com/ldaniel-hm/eml`$_{kb}andit$

[5] Mnih, V., Kavukcuoglu, K., Silver, D., et al. (2015). *Human-level control through deep reinforcement learning.* Nature, 518(7540), 529-533.

[6] Towers, M., Terry, J. K., et al. (2023). *Gymnasium.* Zenodo. Recuperado de: `https://zenodo.org/record/8127025`

[7] *Código fuente y experimentación de la práctica.* GitHub: `https://github.com/Camacho-umu/CamachoMu-ozRuiz`